

MODULE 05/09

Supabase

Backend complet en mode BaaS

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Créer et configurer un projet Supabase de A à Z
 - ✓ Implémenter l'authentification avec Supabase Auth
 - ✓ Créer des tables avec Row Level Security (RLS) pour l'isolation des données
 - ✓ Utiliser Supabase Storage pour les fichiers
 - ✓ Brancher Supabase Realtime pour des mises à jour en temps réel
-

Planning de la semaine

JOUR 1	Supabase Auth	2-3h
• Qu'est-ce que Supabase ? BaaS vs serveur custom • Créer un projet, récupérer les clés API • Inscription/connexion email, gestion des sessions • JWT Supabase : anon key vs service_role key		
JOUR 2	Database et PostgREST	2-3h
• L'interface supabase-js : <code>.from().select().eq()</code> • CRUD complet : insert, update, upsert, delete • Filtres : eq, neq, gt, lt, in, is, ilike • Pagination : range(), order(), limit()		
JOUR 3	Row Level Security (RLS)	2-3h
• Pourquoi RLS ? La sécurité au niveau de la base • Activer RLS sur une table et créer des politiques • <code>auth.uid()</code> et fonctions helper : <code>current_tenant_id()</code> • Tester les politiques en changeant de rôle dans SQL Editor		
JOUR 4	Storage et Realtime	2-3h
• Créer un bucket, politiques d'accès par tenant • Upload / download de fichiers depuis le client • Supabase Realtime : <code>postgres_changes</code> subscriptions • Activer une table dans la publication <code>supabase_realtime</code>		
JOUR 5	Mini-projet de la semaine	3-4h
• Créer une app 'Notes partagées' multi-utilisateurs avec Supabase • Auth email, table notes avec RLS (chaque user voit ses notes)		

- Upload d'une pièce jointe par note (Storage)
- Realtime : les notes se mettent à jour en temps réel entre deux onglets

Concepts clés détaillés

Initialiser le client Supabase

```
// lib/supabase.ts (côté mobile – clé anon publique)
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!
);

// lib/supabaseAdmin.ts (côté backend – service_role privée)
import { createClient } from '@supabase/supabase-js';
import { env } from '../config/env';

export const supabaseAdmin = createClient(
  env.SUPABASE_URL,
  env.SUPABASE_SERVICE_ROLE_KEY,
  { auth: { autoRefreshToken: false, persistSession: false } }
);

// IMPORTANT : service_role bypass RLS – ne JAMAIS exposer côté client
```

Requêtes supabase-js

```
// SELECT avec filtres
const { data, error } = await supabase
  .from('conversations')
  .select('id, platform, customer_name, updated_at')
  .order('updated_at', { ascending: false })
  .limit(20);

// INSERT
const { data: newMsg } = await supabase
  .from('messages')
  .insert({ content: 'Bonjour', direction: 'outbound', conversation_id: id })
  .select()
```

```

.single();

// UPDATE avec condition
await supabase
.from('messages')
.update({ is_read: true })
.eq('conversation_id', conversationId)
.eq('direction', 'inbound')
.eq('is_read', false);

// UPSERT (insert ou update si conflit)
await supabase
.from('push_tokens')
.upsert({ token, platform }, { onConflict: 'token' });

```

Row Level Security (RLS)

```

-- Étape 1 : Activer RLS sur la table
ALTER TABLE public.messages ENABLE ROW LEVEL SECURITY;

-- Étape 2 : Créer une policy SELECT
-- Chaque utilisateur ne voit que les messages de son tenant
CREATE POLICY messages_select ON public.messages
FOR SELECT TO authenticated
USING (tenant_id = public.current_tenant_id());

-- Étape 3 : Policy INSERT
CREATE POLICY messages_insert ON public.messages
FOR INSERT TO authenticated
WITH CHECK (tenant_id = public.current_tenant_id());

-- La fonction helper current_tenant_id()
CREATE OR REPLACE FUNCTION public.current_tenant_id()
RETURNS uuid LANGUAGE sql SECURITY DEFINER
AS $$ SELECT tenant_id FROM public.users WHERE id = auth.uid() $$;

```

Supabase Realtime

```

// S'abonner aux changements d'une table
const channel = supabase
.channel('messages-live')
.on('postgres_changes', {

```

```

event: 'UPDATE',
schema: 'public',
table: 'messages',
filter: `conversation_id=eq.${conversationId}`,
}, (payload) => {
const updated = payload.new as MessageRow;
setMessages(prev => prev.map(m =>
m.id === updated.id ? { ...m, delivery_status: updated.delivery_status } : m
));
})
.subscribe();

// Nettoyage au démontage du composant
return () => { supabase.removeChannel(channel); };

// PRÉREQUIS : activer la table dans la publication
-- ALTER PUBLICATION supabase_realtime ADD TABLE public.messages;

```

■ *anon key = clé publique, incluse dans l'app mobile. Elle est limitée par les RLS policies. service_role key = clé admin, bypass tous les RLS. Ne JAMAIS la mettre dans l'app mobile. Uniquement côté backend (Railway, variables d'environnement serveur).*

Exercices pratiques

Exercice 1 — Auth complète (45 min)

- Créer un projet Supabase, activer le provider Email
- Implémenter inscription + connexion + déconnexion dans une app Expo
- Afficher le nom de l'utilisateur connecté
- Rediriger vers /login si la session expire

Exercice 2 — RLS en pratique (1h)

- Créer une table 'notes' (id, user_id, content, created_at)
- Activer RLS avec policy : chaque user voit seulement ses notes
- Tester dans SQL Editor en changeant de rôle (SET ROLE authenticated)
- Vérifier que l'insertion d'une note avec un faux user_id est bloquée

Ressources pour aller plus loin

- Documentation Supabase : supabase.com/docs
- Supabase + Expo guide : supabase.com/docs/guides/getting-started/tutorials/with-expo-react-native
- Row Level Security : supabase.com/docs/guides/database/postgres/row-level-security

- Supabase Realtime : supabase.com/docs/guides/realtime